

Variational Inference in RL

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Part 1:

Amortized variational inference

Variational inference

$$\log p(x_i) \geq \overbrace{E_{z \sim q_i(z)} [\log p_\theta(x_i|z) + \log p(z)]}^{\mathcal{L}_i(p, q_i)} + \mathcal{H}(q_i)$$

~~$$\theta \leftarrow \arg \max_{\theta} \frac{1}{N} \sum_i \log p_\theta(x_i)$$~~

$$\theta \leftarrow \arg \max_{\theta} \frac{1}{N} \sum_i \mathcal{L}_i(p, q_i)$$

for each x_i (or mini-batch):

calculate $\nabla_{\theta} \mathcal{L}_i(p, q_i)$:

sample $z \sim q_i(z)$

$$\nabla_{\theta} \mathcal{L}_i(p, q_i) \approx \nabla_{\theta} \log p_{\theta}(x_i|z)$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{L}_i(p, q_i)$$

update q_i to maximize $\mathcal{L}_i(p, q_i)$ ← how?

let's say $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$

use gradient $\nabla_{\mu_i} \mathcal{L}_i(p, q_i)$ and $\nabla_{\sigma_i} \mathcal{L}_i(p, q_i)$

gradient ascent on μ_i, σ_i

What's the problem?

for each x_i (or mini-batch):

calculate $\nabla_{\theta} \mathcal{L}_i(p, q_i)$:

sample $z \sim q_i(z)$

$$\nabla_{\theta} \mathcal{L}_i(p, q_i) \approx \nabla_{\theta} \log p_{\theta}(x_i|z)$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{L}_i(p, q_i)$$

update q_i to maximize $\mathcal{L}_i(p, q_i)$

let's say $q_i(z) = \mathcal{N}(\mu_i, \sigma_i)$

use gradient $\nabla_{\mu_i} \mathcal{L}_i(p, q_i)$ and $\nabla_{\sigma_i} \mathcal{L}_i(p, q_i)$

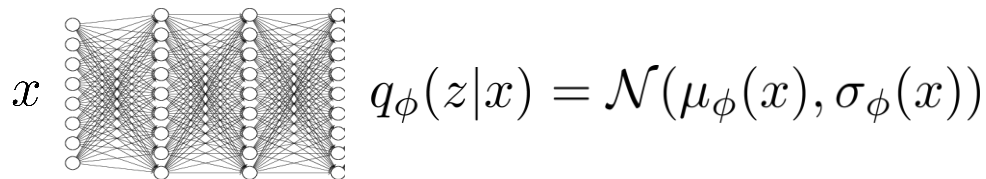
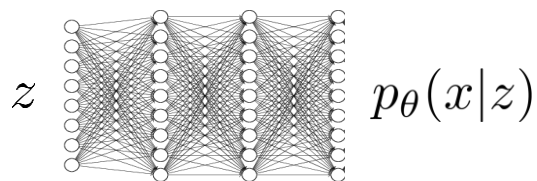
gradient ascent on μ_i, σ_i

How many parameters are there?

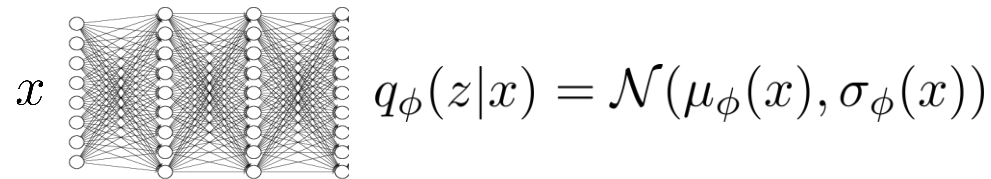
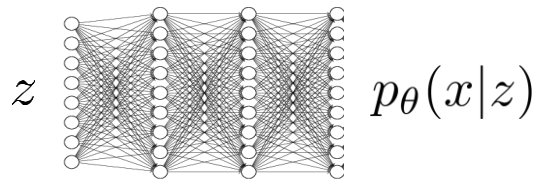
$$|\theta| + (|\mu_i| + |\sigma_i|) \times N$$

intuition: $q_i(z)$ should approximate $p(z|x_i)$

what if we learn a *network* $q_i(z) = q(z|x_i) \approx p(z|x_i)$?



Amortized variational inference



for each x_i (or mini-batch):

calculate $\nabla_\theta \mathcal{L}(p_\theta(x_i|z), q_\phi(z|x_i))$:

sample $z \sim q_\phi(z|x_i)$

$\nabla_\theta \mathcal{L} \approx \nabla_\theta \log p_\theta(x_i|z)$

$\theta \leftarrow \theta + \alpha \nabla_\theta \mathcal{L}$

$\phi \leftarrow \phi + \alpha \nabla_\phi \mathcal{L}$

how do we calculate this?

$$\log p(x_i) \geq \overbrace{E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z) + \log p(z)]}^{\mathcal{L}(p_\theta(x_i|z), q_\phi(z|x_i))} + \mathcal{H}(q_\phi(z|x_i))$$

Amortized variational inference

for each x_i (or mini-batch):

calculate $\nabla_{\theta} \mathcal{L}(p_{\theta}(x_i|z), q_{\phi}(z|x_i))$:

sample $z \sim q_{\phi}(z|x_i)$

$\nabla_{\theta} \mathcal{L} \approx \nabla_{\theta} \log p_{\theta}(x_i|z)$

$\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{L}$

$\phi \leftarrow \phi + \alpha \nabla_{\phi} \mathcal{L}$

$$q_{\phi}(z|x) = \mathcal{N}(\mu_{\phi}(x), \sigma_{\phi}(x))$$

look up formula for
entropy of a Gaussian



$$\mathcal{L}_i = \underbrace{E_{z \sim q_{\phi}(z|x_i)} [\log p_{\theta}(x_i|z) + \log p(z)]}_{J(\phi)} + \mathcal{H}(q_{\phi}(z|x_i))$$

$$J(\phi) = E_{z \sim q_{\phi}(z|x_i)} [r(x_i, z)]$$

can just use policy gradient!

What's wrong with this gradient?

$$\nabla J(\phi) \approx \frac{1}{M} \sum_j \nabla_{\phi} \log q_{\phi}(z_j|x_i) r(x_i, z_j)$$

The reparameterization trick

Is there a better way?

$$\begin{aligned} J(\phi) &= E_{z \sim q_\phi(z|x_i)}[r(x_i, z)] \\ &= E_{\epsilon \sim \mathcal{N}(0,1)}[r(x_i, \mu_\phi(x_i) + \epsilon \sigma_\phi(x_i))] \end{aligned}$$

estimating $\nabla_\phi J(\phi)$:

sample $\epsilon_1, \dots, \epsilon_M$ from $\mathcal{N}(0, 1)$ (a single sample works well!)

$$\nabla_\phi J(\phi) \approx \frac{1}{M} \sum_j \nabla_\phi r(x_i, \mu_\phi(x_i) + \epsilon_j \sigma_\phi(x_i))$$

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x))$$

$$z = \mu_\phi(x) + \epsilon \sigma_\phi(x)$$



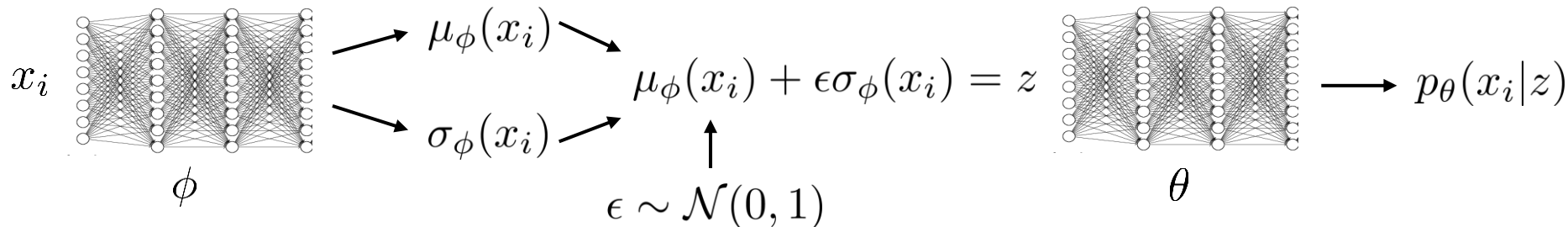
$$\epsilon \sim \mathcal{N}(0, 1)$$

independent of ϕ !

most autodiff software (e.g., PyTorch) will compute this for you!

Another way to look at it...

$$\begin{aligned}\mathcal{L}_i &= E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z) + \log p(z)] + \mathcal{H}(q_\phi(z|x_i)) \\ &= E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z)] + \underbrace{E_{z \sim q_\phi(z|x_i)} [\log p(z)] + \mathcal{H}(q_\phi(z|x_i))}_{-D_{\text{KL}}(q_\phi(z|x_i) \| p(z))} \leftarrow \text{this often has a convenient analytical form (e.g., KL-divergence for Gaussians)} \\ &= E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z)] - D_{\text{KL}}(q_\phi(z|x_i) \| p(z)) \\ &= E_{\epsilon \sim \mathcal{N}(0,1)} [\log p_\theta(x_i | \mu_\phi(x_i) + \epsilon \sigma_\phi(x_i))] - D_{\text{KL}}(q_\phi(z|x_i) \| p(z)) \\ &\approx \log p_\theta(x_i | \mu_\phi(x_i) + \epsilon \sigma_\phi(x_i)) - D_{\text{KL}}(q_\phi(z|x_i) \| p(z))\end{aligned}$$



Reparameterization trick vs. policy gradient

- Policy gradient
 - Can handle both discrete and continuous latent variables
 - High variance, requires multiple samples & small learning rates
- Reparameterization trick
 - Only continuous latent variables
 - Very simple to implement
 - Low variance

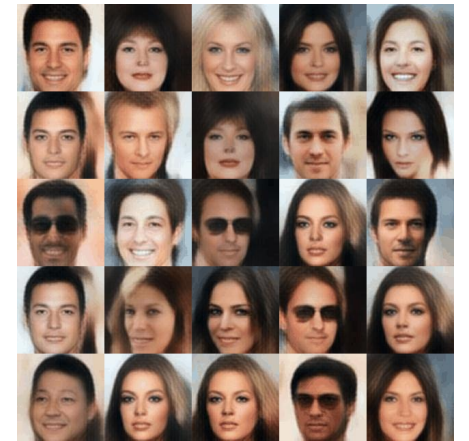
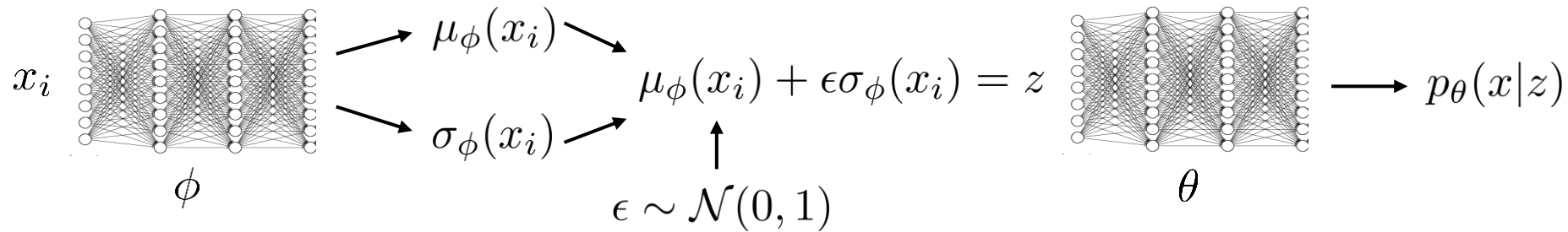
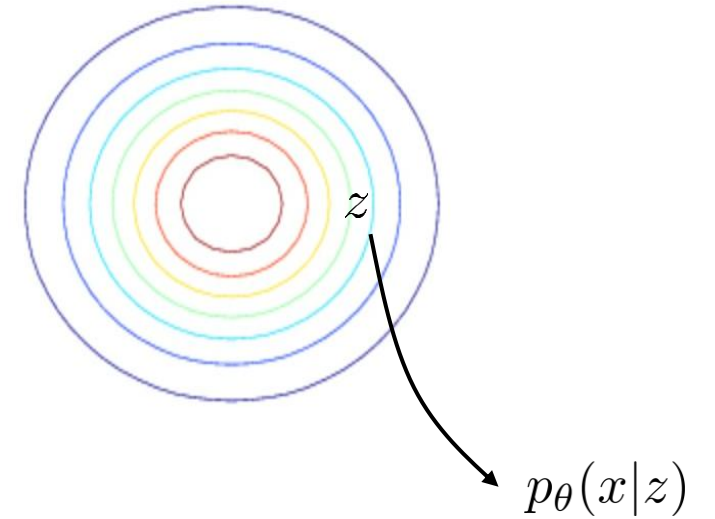
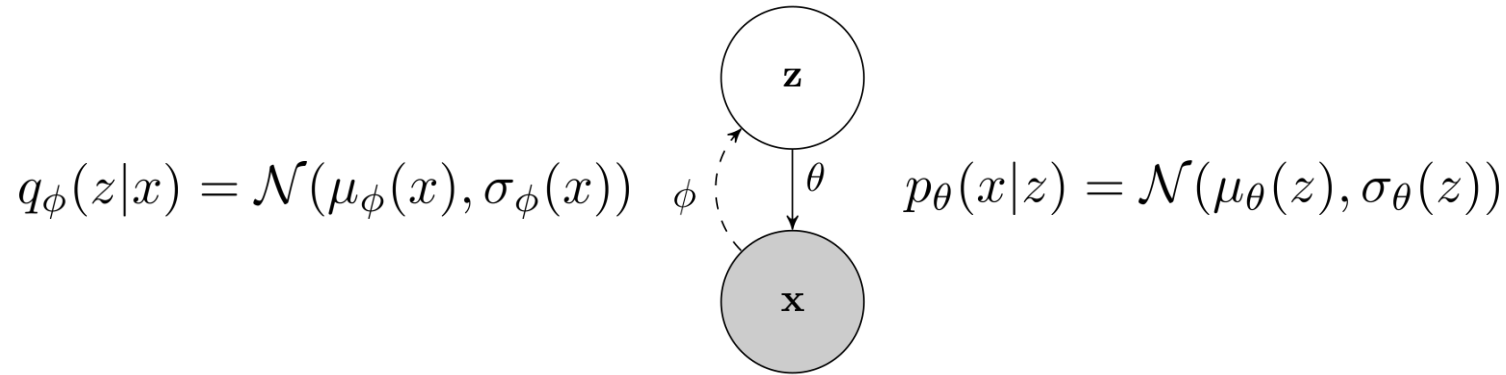
$$J(\phi) \approx \frac{1}{M} \sum_j \nabla_{\phi} \log q_{\phi}(z_j|x_i) r(x_i, z_j)$$

$$\nabla_{\phi} J(\phi) \approx \frac{1}{M} \sum_j \nabla_{\phi} r(x_i, \mu_{\phi}(x_i) + \epsilon_j \sigma_{\phi}(x_i))$$

Part 2:

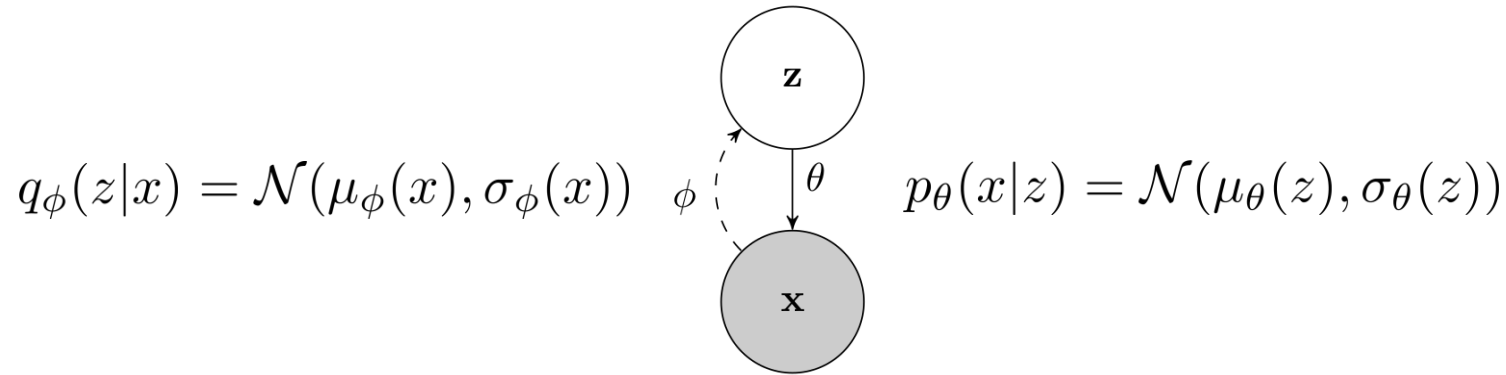
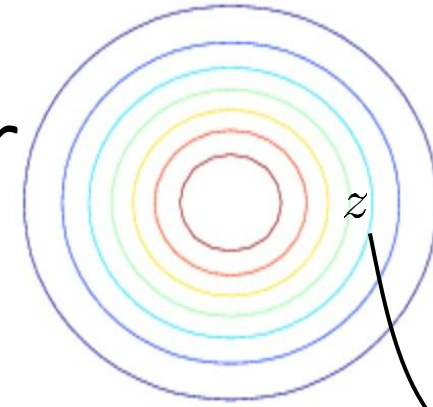
Generative models

The *variational* autoencoder



$$\max_{\theta, \phi} \frac{1}{N} \sum_i \log p_\theta(x_i | \mu_\phi(x_i) + \epsilon \sigma_\phi(x_i)) - D_{\text{KL}}(q_\phi(z|x_i) \| p(z))$$

Using the variational autoencoder



$$p(x) = \int p(x|z)p(z)dz$$

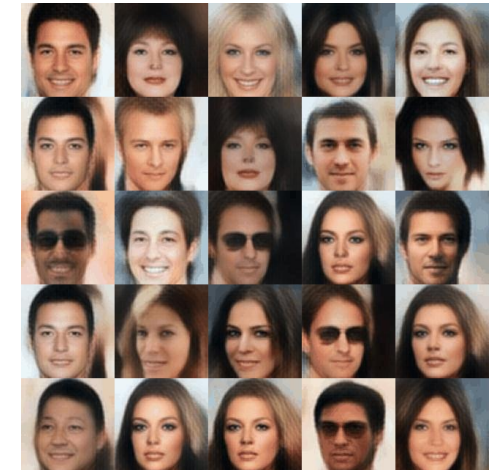
why does this work?

sampling:

$$z \sim p(z)$$

$$x \sim p(x|z)$$

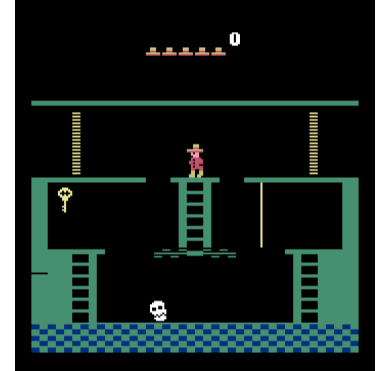
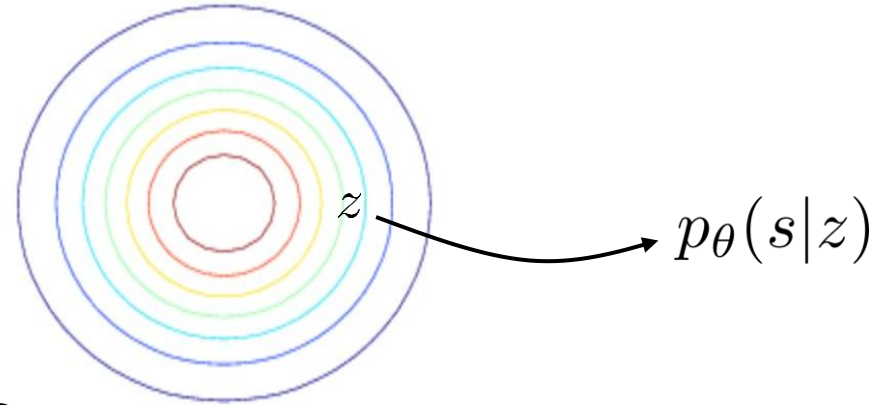
$$\mathcal{L}_i = E_{z \sim q_\phi(z|x_i)} [\log p_\theta(x_i|z)] - D_{\text{KL}}(q_\phi(z|x_i) || p(z))$$



Example applications

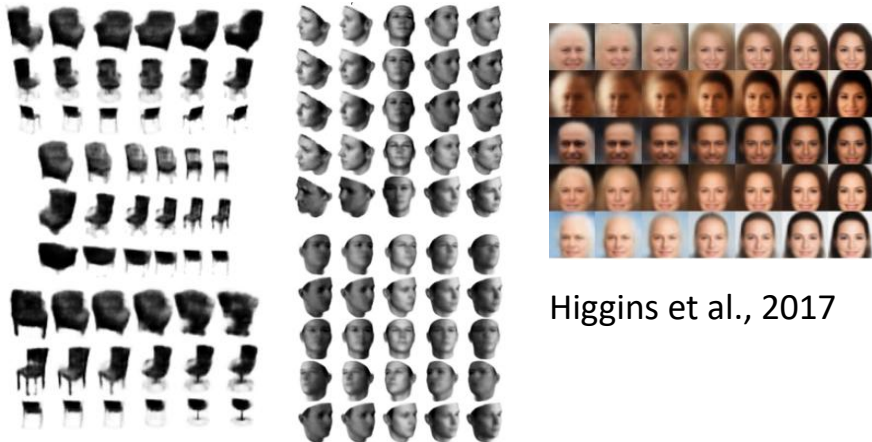
Representation learning

z is a *representation* of s



1. Train VAE on states in replay buffer $\mathcal{R}$
2. Run RL, using z as the state instead of s

Why is this a good idea?



Higgins et al., 2017

Sample algorithm

1. Collect transition (s, a, s', r) , add to $\mathcal{R}$
2. Update $p_\theta(s|z)$ and $q_\phi(z|s)$ w/ batch from $\mathcal{R}$
3. Update $Q(z, a)$ w/ batch from $\mathcal{R}$

This also provides a great way to use prior data!

Conditional models

$$\mathcal{L}_i = E_{z \sim q_\phi(z|x_i, y_i)} [\log p_\theta(y_i|x_i, z) + \log p(z|x_i)] + \mathcal{H}(q_\phi(z|x_i, y_i))$$

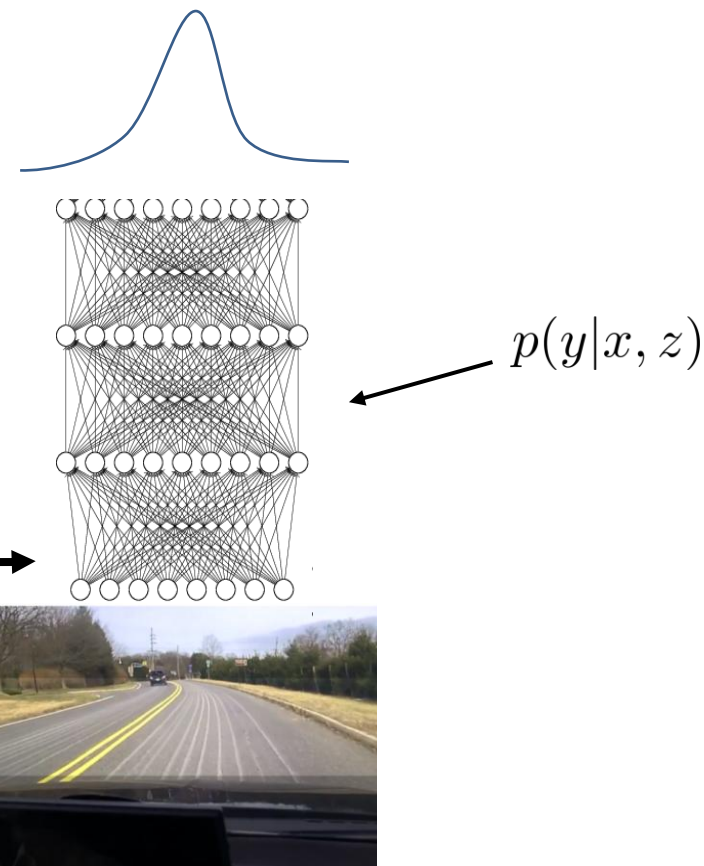
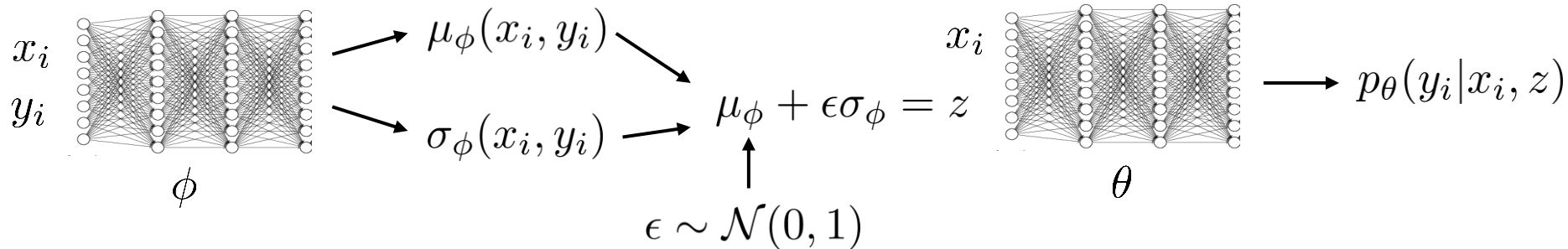
just like before, only now generating y_i
and *everything* is conditioned on x_i

at test time:

$$z \sim p(z|x_i)$$

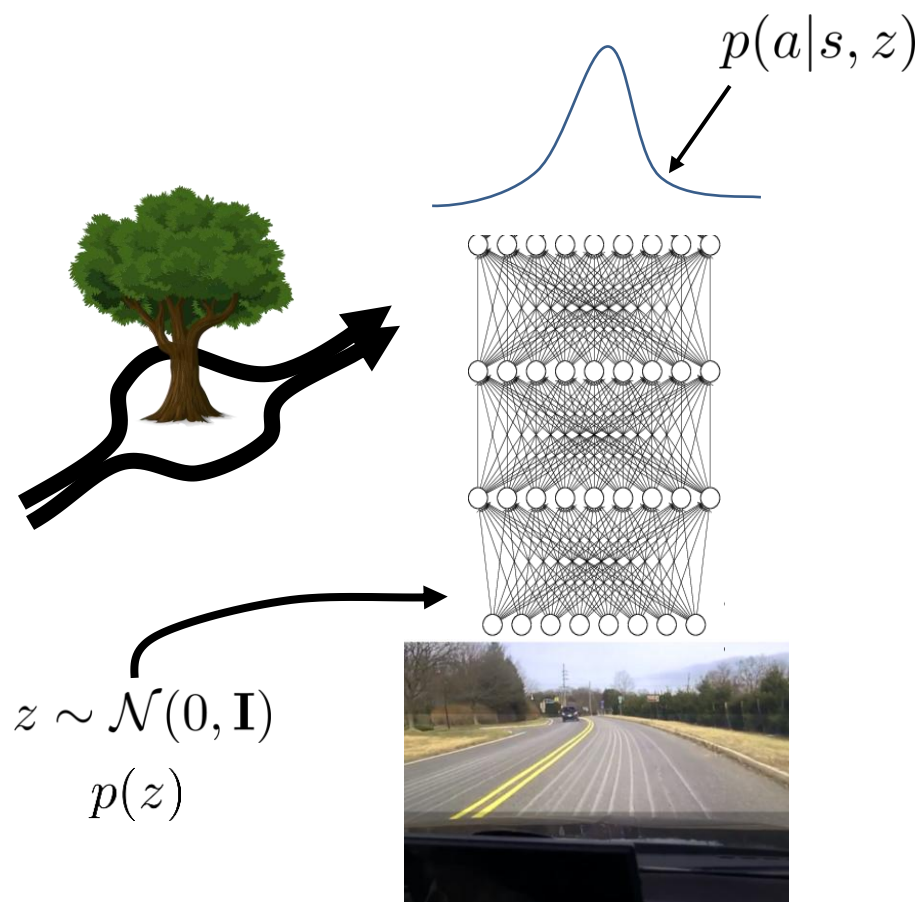
$$y \sim p(y|x_i, z)$$

can *optionally* depend on x



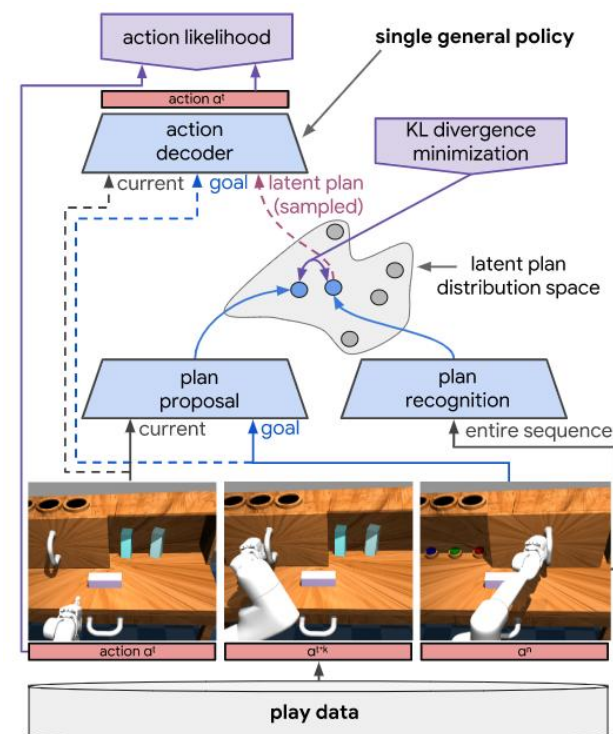
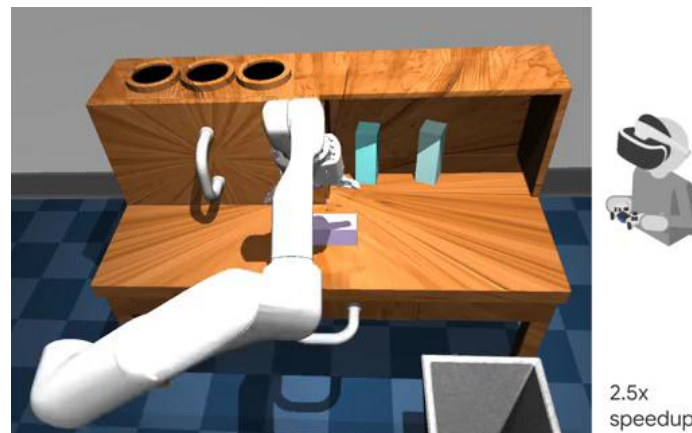
Example applications

Multimodal imitation learning



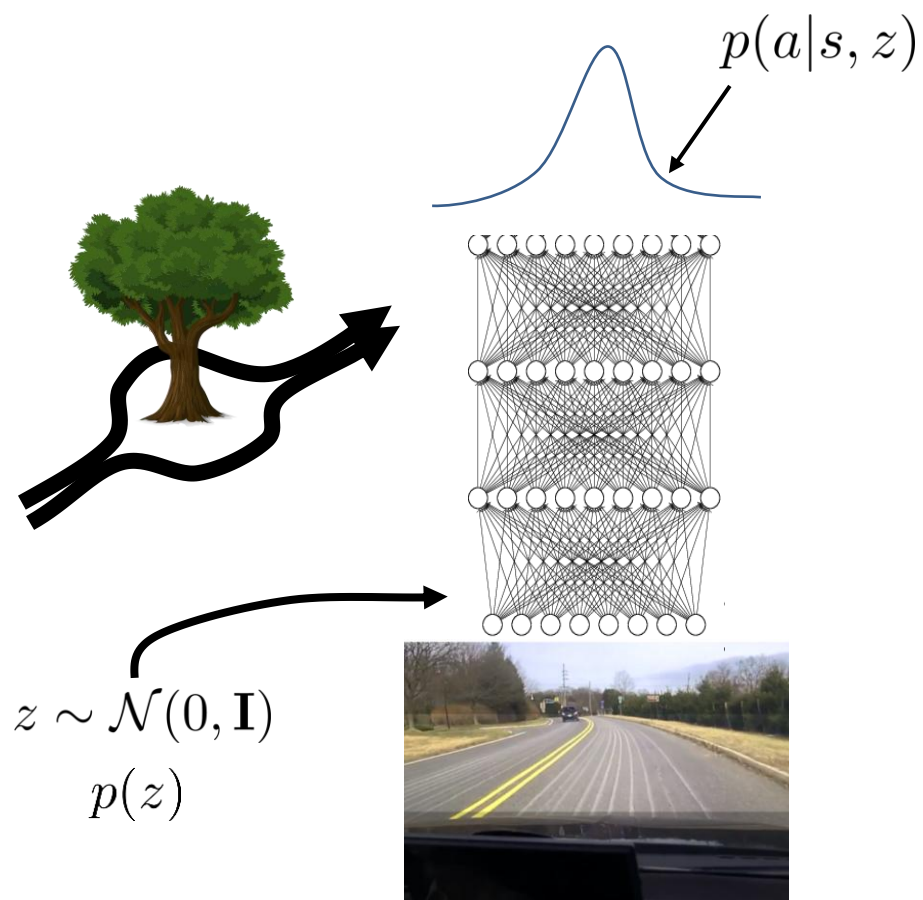
Learning Latent Plans from Play

COREY LYNCH Google Brain MOHI KHANSARI Google X TED XIAO Google Brain VIKASH KUMAR Google Brain JONATHAN TOMPSON Google Brain SERGEY LEVINE Google Brain PIERRE SERMANET Google Brain



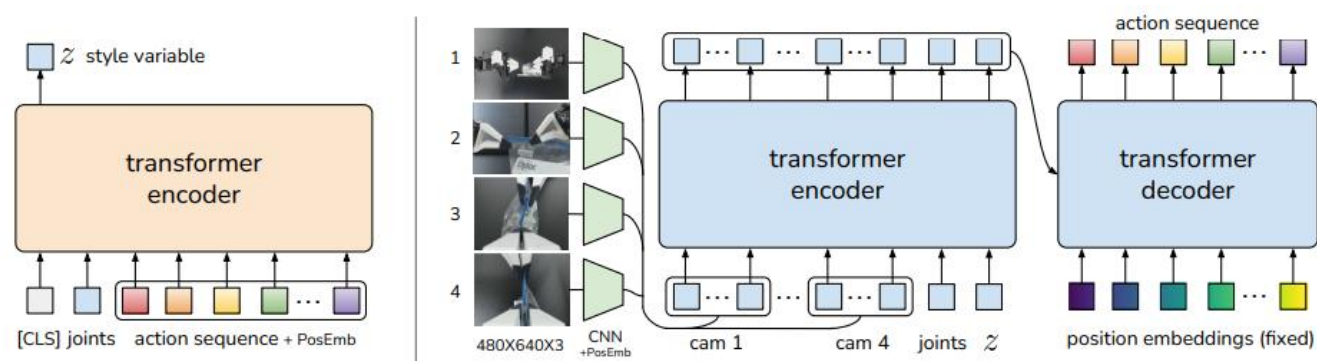
Example applications

Multimodal imitation learning



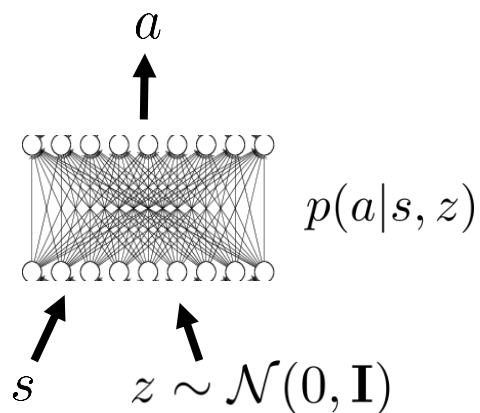
Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware

Tony Z. Zhao¹ Vikash Kumar³ Sergey Levine² Chelsea Finn¹
¹ Stanford University ² UC Berkeley ³ Meta



Relationship to diffusion and flow matching

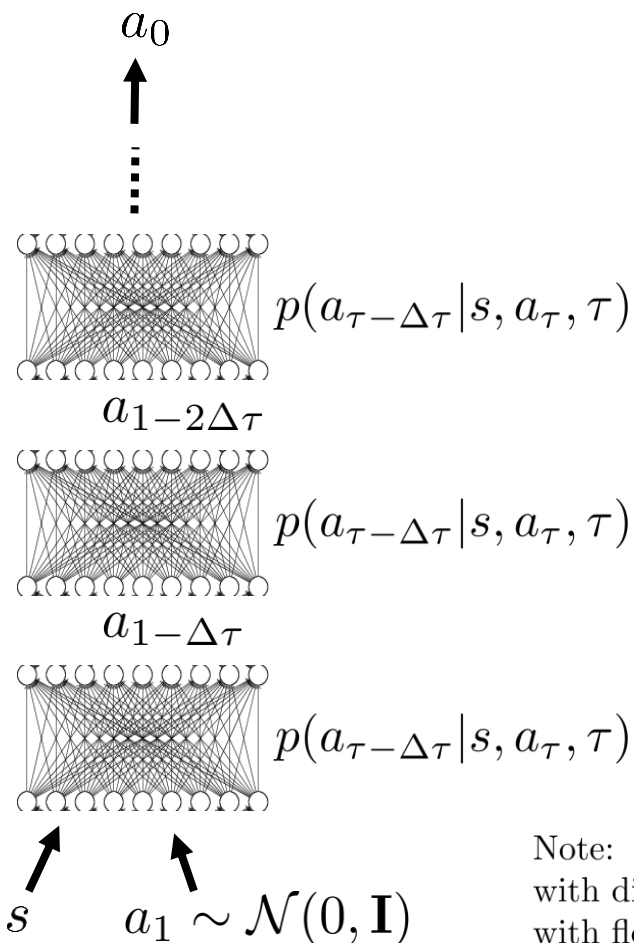
conditional VAE



$$\log p(a|s) \geq$$

$$E_{z \sim q(z|s)} [\log p(a|s, z)] - D_{\text{KL}}(q(z|s), p(z))$$

diffusion or flow



$$\log p(a_0|s) \geq$$

$$E_{a_{\Delta\tau:1} \sim q(a_{\Delta\tau:1}|s)} [\log p(a_0|s, a_{\Delta\tau:1})]$$

$$-D_{\text{KL}}(q(a_{\Delta\tau:1}|a_0), p(a_{\Delta\tau:1}))$$

forward
(noising)
process

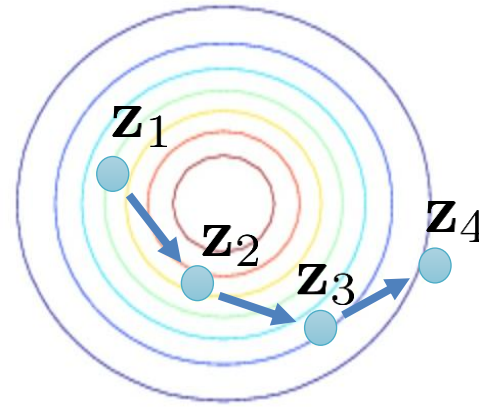
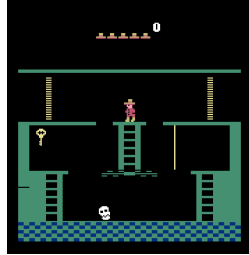
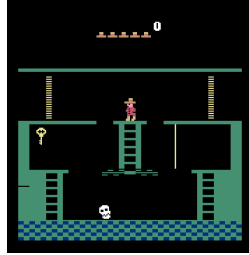
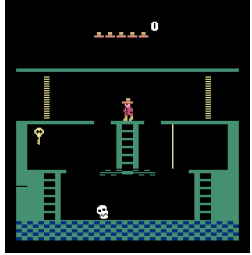
reverse
(denoising)
process

diffusion can be thought of
as a hierarchical VAE

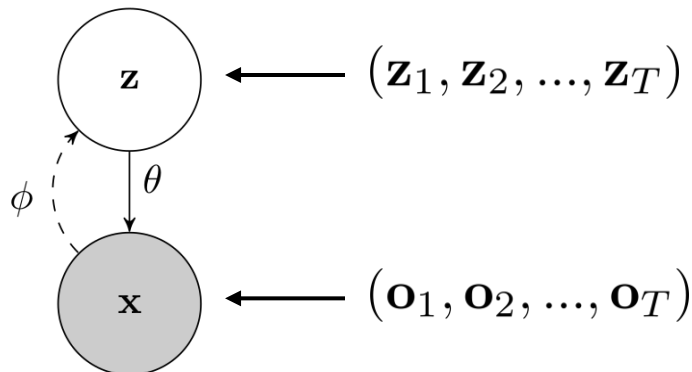
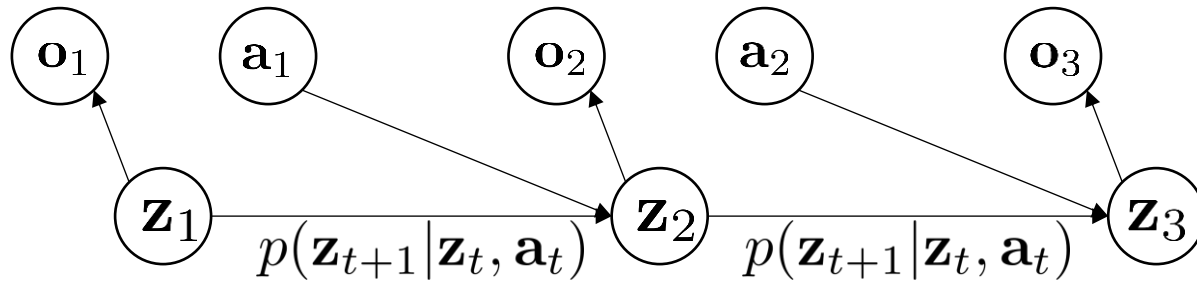
Note:

with diffusion (DDPM), noise is a_1
with flow matching, noise is a_0

State space models



we'll learn more about this when we discuss model-based RL!



What is the **prior**?

What is the **decoder**?

What is the **encoder**?

$$p(\mathbf{z}) = p(\mathbf{z}_1) \prod_t p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$$

$\mathcal{N}(0, \mathbf{I})$ (prior) points to $p(\mathbf{z}_1)$
 learned points to $p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$

$$p_\theta(\mathbf{o} | \mathbf{z}) = \prod_t p(\mathbf{o}_t | \mathbf{z}_t)$$

$$q_\phi(\mathbf{z} | \mathbf{o}) = \prod_t q_\phi(\mathbf{z}_t | \mathbf{o}_{1:t})$$

Intermission

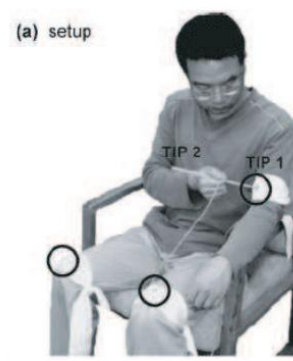
Part 3:

Control as inference

Optimal Control as a Model of Human Behavior



Mombaur et al. '09



Li & Todorov '06



Ziebart '08

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)$$

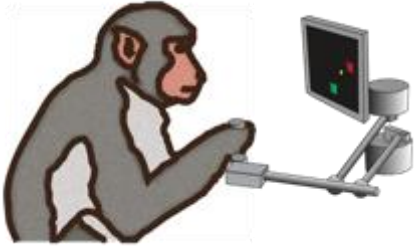
$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$

$$\pi = \arg \max_{\pi} E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t), \mathbf{a}_t \sim \pi(\mathbf{a}_t | \mathbf{s}_t)} [r(\mathbf{s}_t, \mathbf{a}_t)]$$

$$\mathbf{a}_t \sim \pi(\mathbf{a}_t | \mathbf{s}_t)$$

optimize this to explain the data

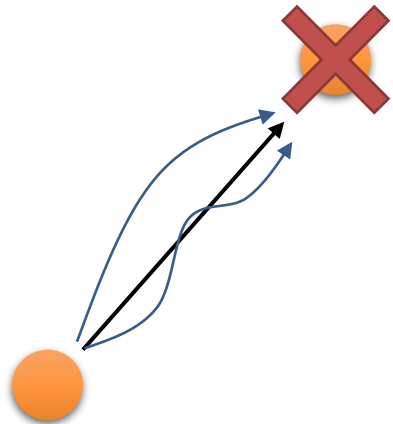
What if the data is **not** optimal?



some mistakes matter more than others!

behavior is **stochastic**

but good behavior is still the most likely



A probabilistic graphical model of decision making

~~$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t)$$

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$~~

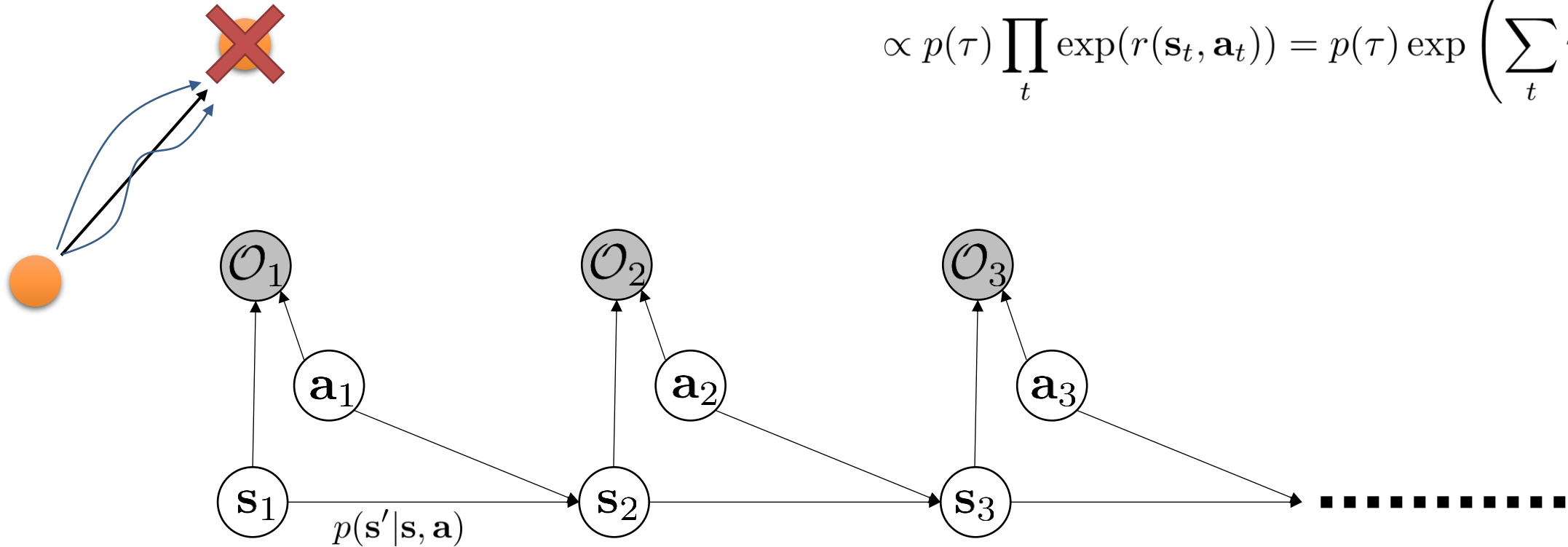
$$p(\underbrace{\mathbf{s}_{1:T}, \mathbf{a}_{1:T}}_{\tau}) = ?? \quad \text{no assumption of optimal behavior!}$$

$$p(\tau | \mathcal{O}_{1:T})$$

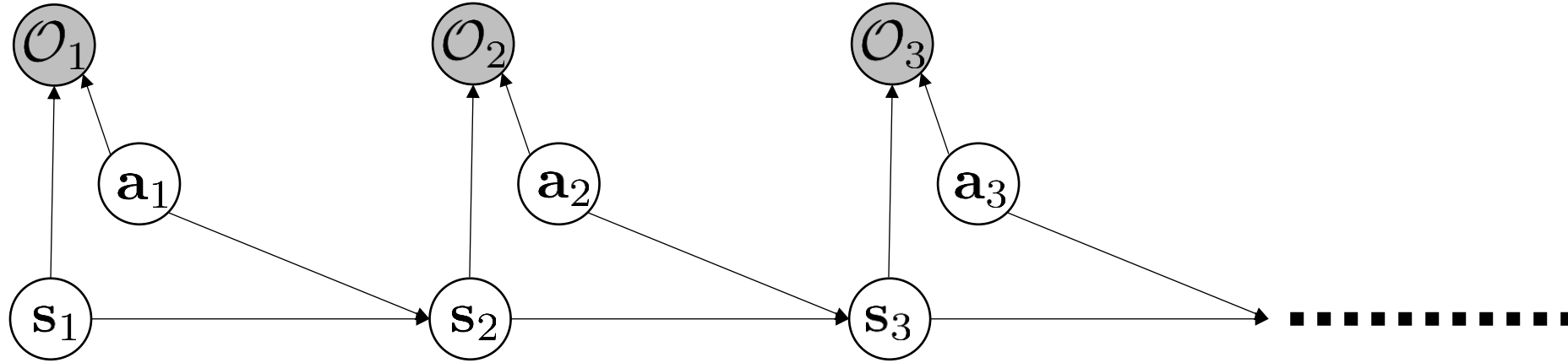
$$p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) = \exp(r(\mathbf{s}_t, \mathbf{a}_t))$$

$$p(\tau | \mathcal{O}_{1:T}) = \frac{p(\tau, \mathcal{O}_{1:T})}{p(\mathcal{O}_{1:T})}$$

$$\propto p(\tau) \prod_t \exp(r(\mathbf{s}_t, \mathbf{a}_t)) = p(\tau) \exp\left(\sum_t r(\mathbf{s}_t, \mathbf{a}_t)\right)$$

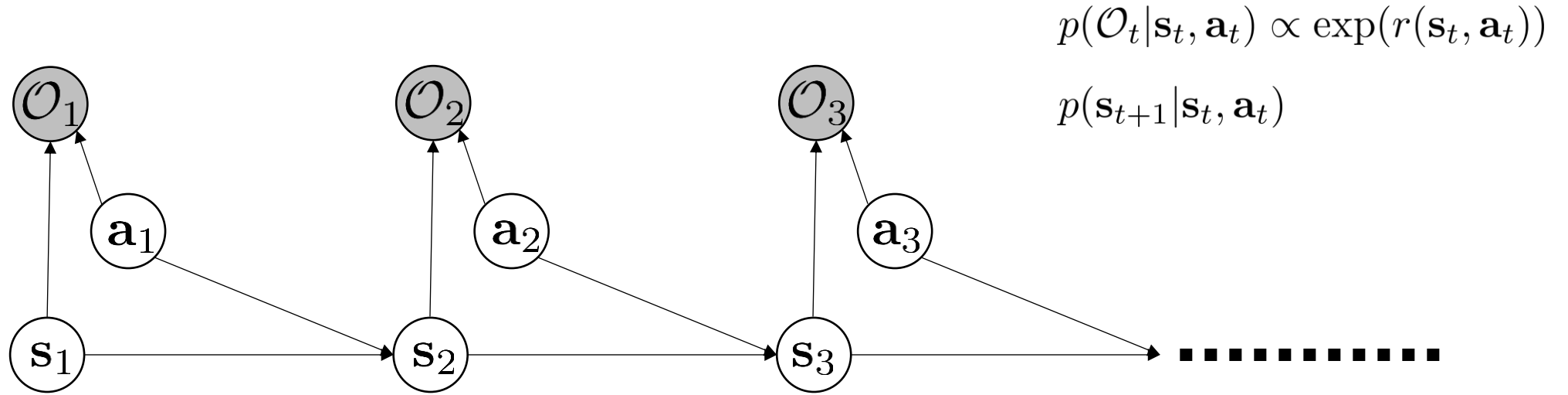


Why is this interesting?



- Can model suboptimal behavior (important for inverse RL)
- Can apply inference algorithms to solve control and planning problems
- Provides an explanation for why stochastic behavior might be preferred (useful for exploration and transfer learning)

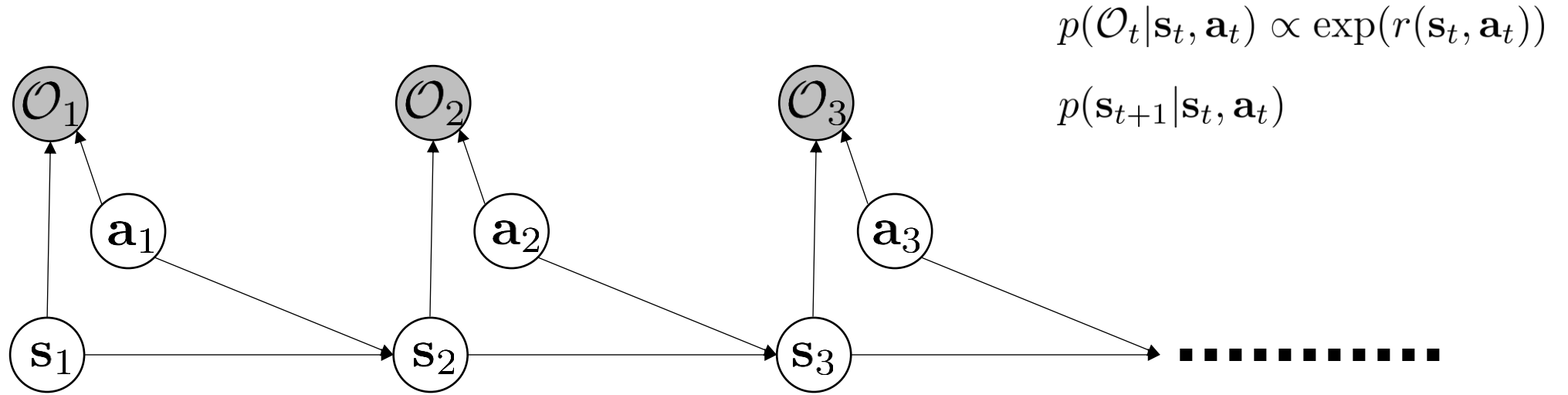
Inference = planning



how to do inference?

1. compute backward messages $\beta_t(s_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | s_t, \mathbf{a}_t)$
2. compute policy $p(\mathbf{a}_t | s_t, \mathcal{O}_{1:T})$
3. compute forward messages $\alpha_t(s_t) = p(s_t | \mathcal{O}_{1:t-1})$

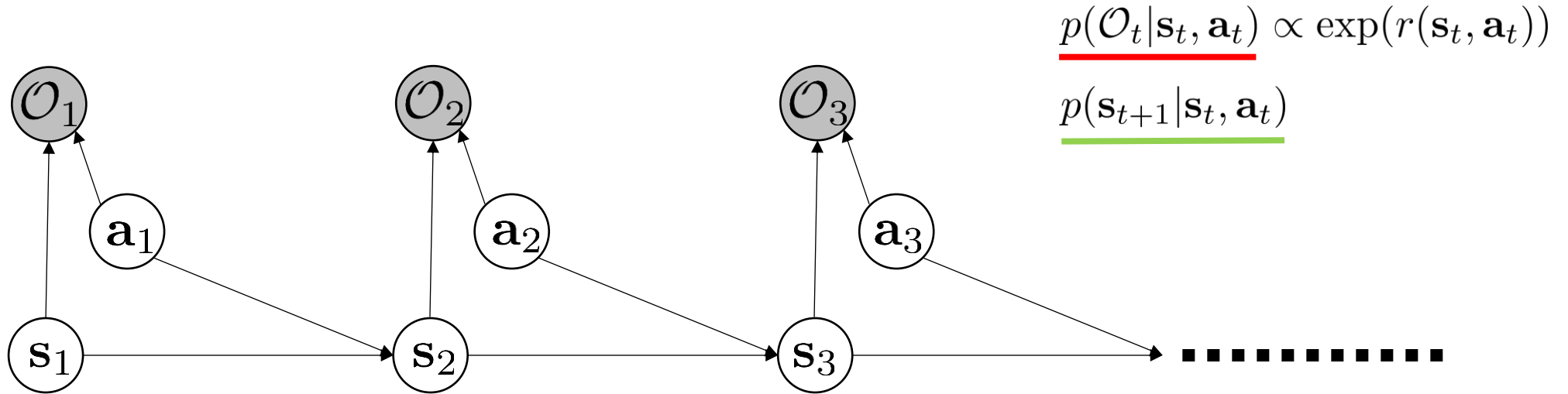
Inference = planning



how to do inference?

1. compute backward messages $\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t)$
2. compute policy $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T})$
3. compute forward messages $\alpha_t(\mathbf{s}_t) = p(\mathbf{s}_t | \mathcal{O}_{1:t-1})$

Backward messages



$$\begin{aligned}
 \beta_t(\mathbf{s}_t, \mathbf{a}_t) &= p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t) \\
 &= \int p(\mathcal{O}_{t:T}, \mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t) d\mathbf{s}_{t+1} \\
 &= \int p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}) \underline{p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} \underline{p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t)} d\mathbf{s}_{t+1} \xrightarrow{\text{for } t = T-1 \text{ to } 1:} \beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\beta_{t+1}(\mathbf{s}_{t+1})] \\
 p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}) &= \int \underline{p(\mathcal{O}_{t+1:T} | \mathbf{s}_{t+1}, \mathbf{a}_{t+1})} \cancel{p(\mathbf{a}_{t+1} | \mathbf{s}_{t+1})} d\mathbf{a}_{t+1} \xrightarrow{\quad} \beta_t(\mathbf{s}_t) = E_{\mathbf{a}_t \sim p(\mathbf{a}_t | \mathbf{s}_t)} [\beta_t(\mathbf{s}_t, \mathbf{a}_t)] \\
 &\quad \beta_t(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) \quad \uparrow \\
 &\quad \text{which actions are likely } a \text{ priori} \\
 &\quad \text{(assume uniform for now)}
 \end{aligned}$$

A closer look at the backward pass

for $t = T - 1$ to 1:

$$\underline{\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\beta_{t+1}(\mathbf{s}_{t+1})]}$$

$$\underline{\beta_t(\mathbf{s}_t) = E_{\mathbf{a}_t \sim p(\mathbf{a}_t | \mathbf{s}_t)} [\beta_t(\mathbf{s}_t, \mathbf{a}_t)]}$$

$$\text{let } V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t)$$

$$\text{let } Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t)$$

$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) d\mathbf{a}_t$$

$$V_t(\mathbf{s}_t) \rightarrow \max_{\mathbf{a}_t} Q_t(\mathbf{s}_t, \mathbf{a}_t) \text{ as } Q_t(\mathbf{s}_t, \mathbf{a}_t) \text{ gets bigger!}$$

value iteration algorithm:

- 
1. set $Q(\mathbf{s}, \mathbf{a}) \leftarrow r(\mathbf{s}, \mathbf{a}) + \gamma E[V(\mathbf{s}')]]$
 2. set $V(\mathbf{s}) \leftarrow \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$

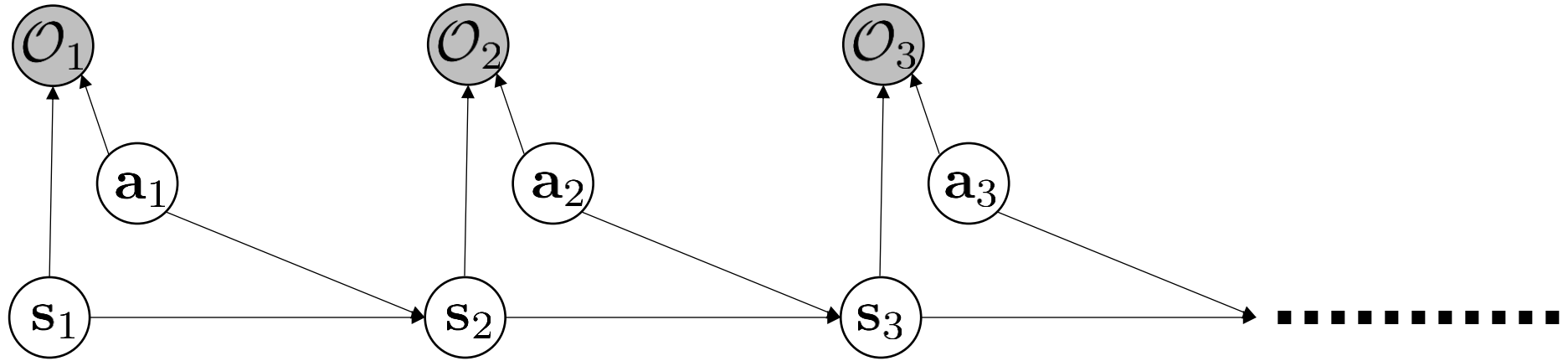
“optimistic” transition
(not a good idea!)

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \overbrace{\log E[\exp(V_{t+1}(\mathbf{s}_{t+1}))]}$$

$$\text{deterministic transition: } Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + V_{t+1}(\mathbf{s}_{t+1})$$

we'll come back to the stochastic case later!

Backward pass summary



$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t)$$

probability that we can be optimal at steps t through T
given that we take action $\mathbf{a}_t$ in state $\mathbf{s}_t$

for $t = T - 1$ to 1:

$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_t | \mathbf{s}_t, \mathbf{a}_t) E_{\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)} [\beta_{t+1}(\mathbf{s}_{t+1})] \quad \text{compute recursively from } t = T \text{ to } t = 1$$

$$\beta_t(\mathbf{s}_t) = E_{\mathbf{a}_t \sim p(\mathbf{a}_t | \mathbf{s}_t)} [\beta_t(\mathbf{s}_t, \mathbf{a}_t)]$$

$$\text{let } V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t)$$

$$\text{let } Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t)$$

log of β_t is “ Q -function-like”

The action prior

remember this?

$$p(\mathcal{O}_{t+1:T}|\mathbf{s}_{t+1}) = \int \underbrace{p(\mathcal{O}_{t+1:T}|\mathbf{s}_{t+1}, \mathbf{a}_{t+1})}_{\beta_t(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})} \cancel{p(\mathbf{a}_{t+1}|\mathbf{s}_{t+1})} d\mathbf{a}_{t+1}$$

↑
("soft max")

what if the action prior is not uniform?

$$V(\mathbf{s}_t) = \log \int \exp(Q(\mathbf{s}_t, \mathbf{a}_t) + \log p(\mathbf{a}_t|\mathbf{s}_t)) \mathbf{a}_t$$

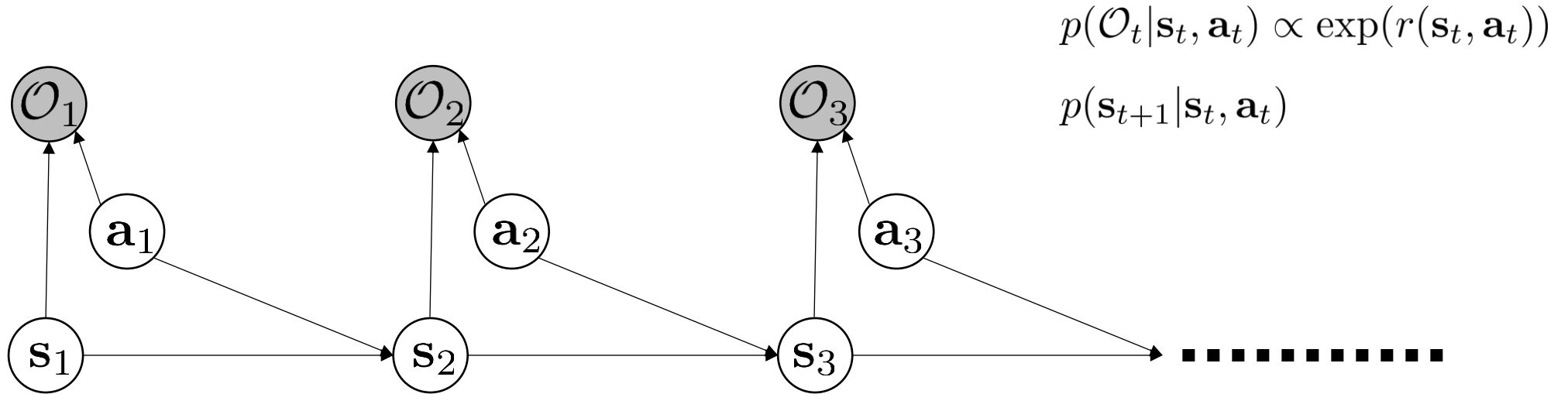
$$Q(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log E[\exp(V(\mathbf{s}_{t+1}))]$$

let $\tilde{Q}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log p(\mathbf{a}_t|\mathbf{s}_t) + \log E[\exp(V(\mathbf{s}_{t+1}))]$

$$V(\mathbf{s}_t) = \log \int \exp(\tilde{Q}(\mathbf{s}_t, \mathbf{a}_t)) \mathbf{a}_t \quad \Leftrightarrow \quad V(\mathbf{s}_t) = \log \int \exp(Q(\mathbf{s}_t, \mathbf{a}_t) + \log p(\mathbf{a}_t|\mathbf{s}_t)) \mathbf{a}_t$$

can **always** fold the action prior into the reward! uniform action prior
can be assumed without loss of generality

Policy computation



2. compute policy $p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T})$

$$\beta_t(\mathbf{s}_t, \mathbf{a}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\beta_t(\mathbf{s}_t) = p(\mathcal{O}_{t:T} | \mathbf{s}_t)$$

$$p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{1:T}) = \pi(\mathbf{a}_t | \mathbf{s}_t)$$

$$= p(\mathbf{a}_t | \mathbf{s}_t, \mathcal{O}_{t:T})$$

$$= \frac{p(\mathbf{a}_t, \mathbf{s}_t | \mathcal{O}_{t:T})}{p(\mathbf{s}_t | \mathcal{O}_{t:T})}$$

$$= \frac{p(\mathcal{O}_{t:T} | \mathbf{a}_t, \mathbf{s}_t) p(\mathbf{a}_t, \mathbf{s}_t) / \cancel{p(\mathcal{O}_{t:T})}}{p(\mathcal{O}_{t:T} | \mathbf{s}_t) p(\mathbf{s}_t) / \cancel{p(\mathcal{O}_{t:T})}}$$

$$= \frac{p(\mathcal{O}_{t:T} | \mathbf{a}_t, \mathbf{s}_t)}{p(\mathcal{O}_{t:T} | \mathbf{s}_t)} \frac{p(\mathbf{a}_t, \mathbf{s}_t)}{p(\mathbf{s}_t)} = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)} \cancel{p(\mathbf{a}_t | \mathbf{s}_t)}$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)}$$

Policy computation with value functions

for $t = T - 1$ to 1:

$$Q_t(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \log E[\exp(V_{t+1}(\mathbf{s}_{t+1}))]$$

$$V_t(\mathbf{s}_t) = \log \int \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t)) \mathbf{a}_t$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \frac{\beta_t(\mathbf{s}_t, \mathbf{a}_t)}{\beta_t(\mathbf{s}_t)} \quad \begin{array}{l} V_t(\mathbf{s}_t) = \log \beta_t(\mathbf{s}_t) \\ Q_t(\mathbf{s}_t, \mathbf{a}_t) = \log \beta_t(\mathbf{s}_t, \mathbf{a}_t) \end{array}$$

$$\pi(\mathbf{a}_t | \mathbf{s}_t) = \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t) - V_t(\mathbf{s}_t)) = \exp(A_t(\mathbf{s}_t, \mathbf{a}_t))$$

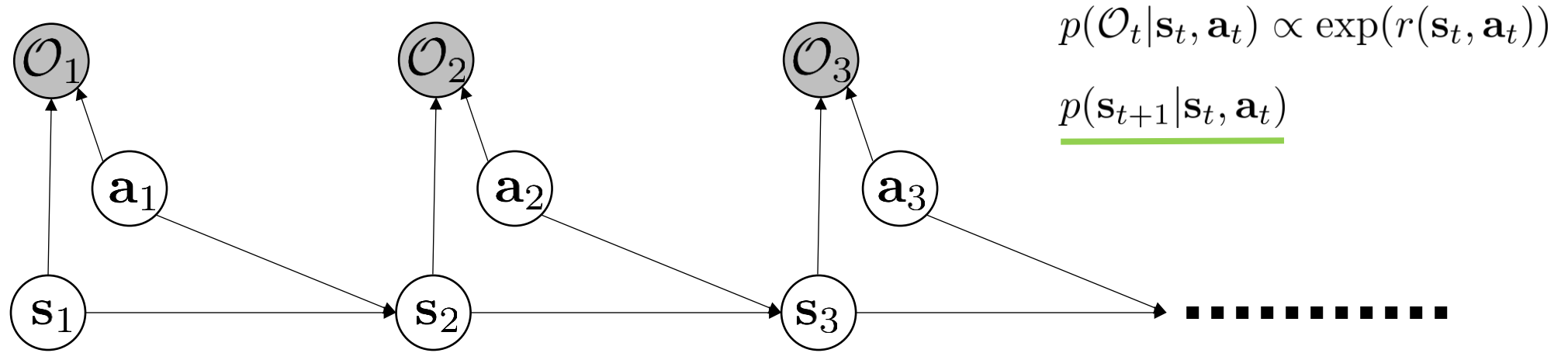
Policy computation summary

$$\pi(\mathbf{a}_t|\mathbf{s}_t) = \exp(Q_t(\mathbf{s}_t, \mathbf{a}_t) - V_t(\mathbf{s}_t)) = \exp(A_t(\mathbf{s}_t, \mathbf{a}_t))$$

with temperature: $\pi(\mathbf{a}_t|\mathbf{s}_t) = \exp(\frac{1}{\alpha}Q_t(\mathbf{s}_t, \mathbf{a}_t) - \frac{1}{\alpha}V_t(\mathbf{s}_t)) = \exp(\frac{1}{\alpha}A_t(\mathbf{s}_t, \mathbf{a}_t))$

- Natural interpretation: better actions are more probable
- Random tie-breaking
- Analogous to Boltzmann exploration
- Approaches greedy policy as temperature decreases

Forward messages



$$\alpha_1(\mathbf{s}_1) = p(\mathbf{s}_1) \text{ (usually known)}$$

$$\alpha_t(\mathbf{s}_t) = p(\mathbf{s}_t | \mathcal{O}_{1:t-1})$$

$$= \int p(\mathbf{s}_t, \mathbf{s}_{t-1}, \mathbf{a}_{t-1} | \mathcal{O}_{1:t-1}) d\mathbf{s}_{t-1} d\mathbf{a}_{t-1} = \int p(\mathbf{s}_t | \mathbf{s}_{t-1}, \mathbf{a}_{t-1}, \cancel{\mathcal{O}_{1:t-1}}) p(\mathbf{a}_{t-1} | \mathbf{s}_{t-1}, \mathcal{O}_{1:t-1}) p(\mathbf{s}_{t-1} | \mathcal{O}_{1:t-1}) d\mathbf{s}_{t-1} d\mathbf{a}_{t-1}$$

$$= \int \underline{p(\mathbf{s}_t | \mathbf{s}_{t-1}, \mathbf{a}_{t-1})} p(\mathbf{a}_{t-1} | \mathbf{s}_{t-1}, \mathcal{O}_{t-1}) p(\mathbf{s}_{t-1} | \mathcal{O}_{1:t-1}) d\mathbf{s}_{t-1} d\mathbf{a}_{t-1}$$

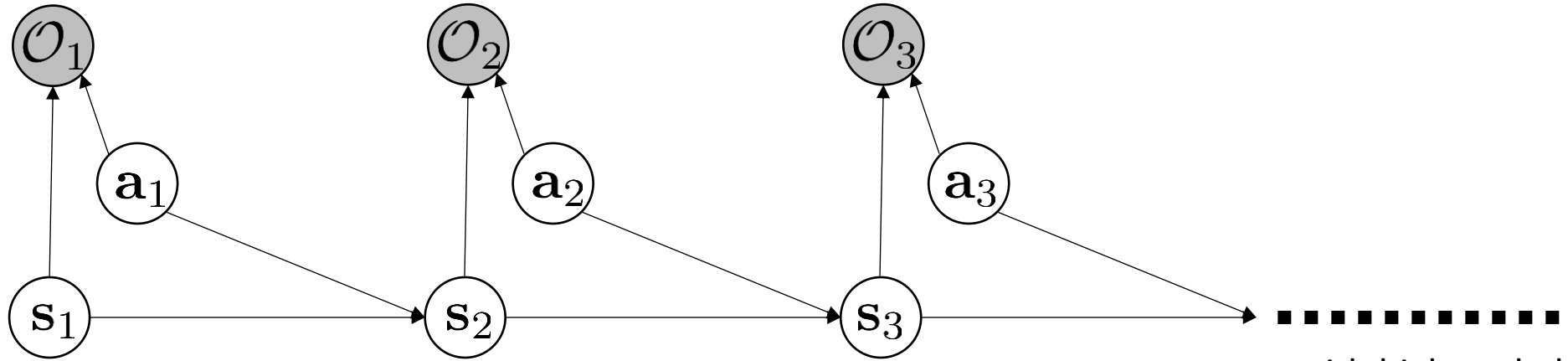
$$p(\mathbf{a}_{t-1} | \mathbf{s}_{t-1}, \mathcal{O}_{t-1}) p(\mathbf{s}_{t-1} | \mathcal{O}_{1:t-1}) = \frac{p(\mathcal{O}_{t-1} | \mathbf{s}_{t-1}, \mathbf{a}_{t-1}) p(\mathbf{a}_{t-1} | \mathbf{s}_{t-1})}{p(\cancel{\mathcal{O}_{t-1}} | \mathbf{s}_{t-1})} \frac{\alpha_{t-1}(\mathbf{s}_{t-1}) p(\mathcal{O}_{t-1} | \cancel{\mathbf{s}_{t-1}}) p(\mathbf{s}_{t-1} | \underline{\mathcal{O}_{1:t-2}})}{p(\mathcal{O}_{t-1} | \mathcal{O}_{1:t-2})}$$

what if we want $p(\mathbf{s}_t | \mathcal{O}_{1:T})$?

$$\beta_t(\mathbf{s}_t)$$

$$p(\mathbf{s}_t | \mathcal{O}_{1:T}) = \frac{p(\mathbf{s}_t, \mathcal{O}_{1:T})}{p(\mathcal{O}_{1:T})} = \frac{\overset{\beta_t(\mathbf{s}_t)}{p(\mathcal{O}_{t:T} | \mathbf{s}_t)} p(\mathbf{s}_t, \mathcal{O}_{1:t-1})}{p(\mathcal{O}_{1:T})} \propto \beta_t(\mathbf{s}_t) \underline{p(\mathbf{s}_t | \mathcal{O}_{1:t-1})} \cancel{p(\mathcal{O}_{1:t-1})} \propto \beta_t(\mathbf{s}_t) \alpha_t(\mathbf{s}_t)$$

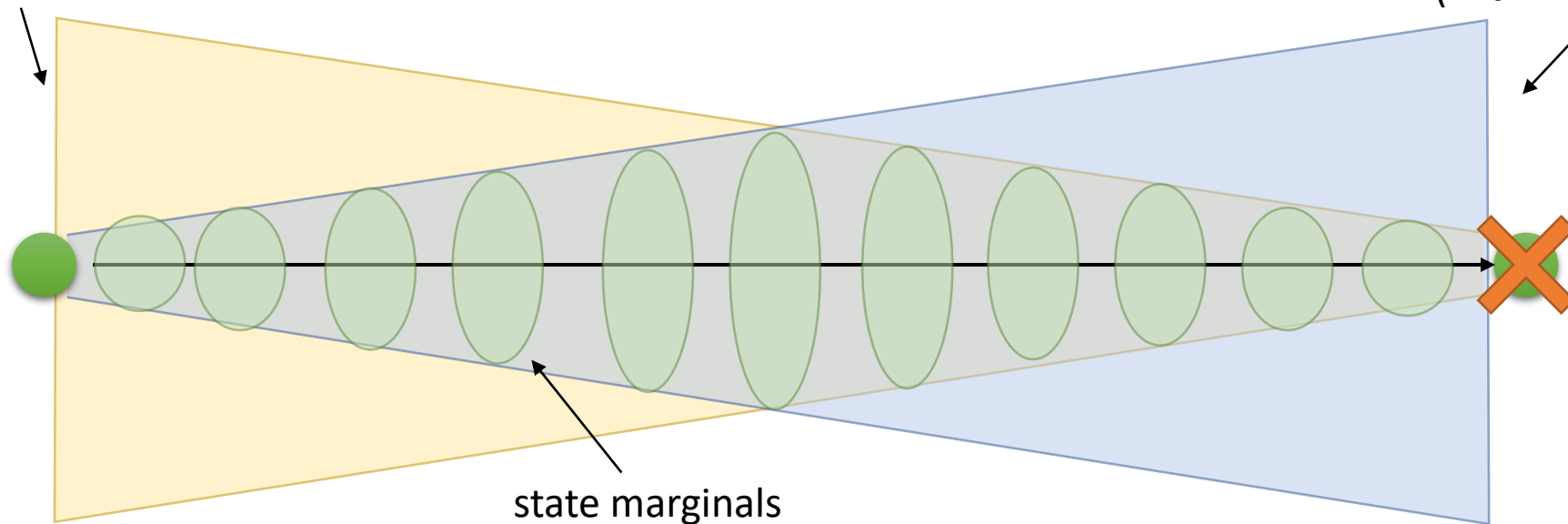
Forward/backward message intersection



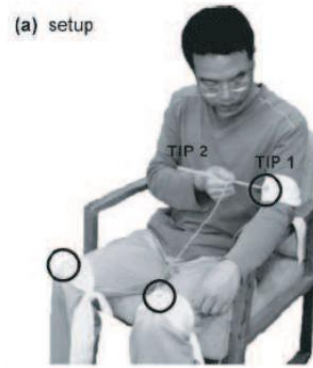
states with high probability of reaching goal

$$p(s_t) \propto \beta_t(s_t) \alpha_t(s_t)$$

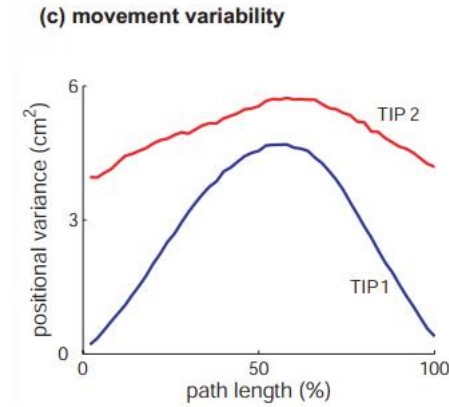
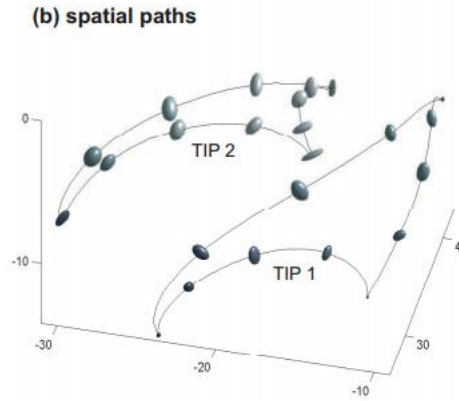
states with high probability of being reached from initial state (with high reward)



Forward/backward message intersection



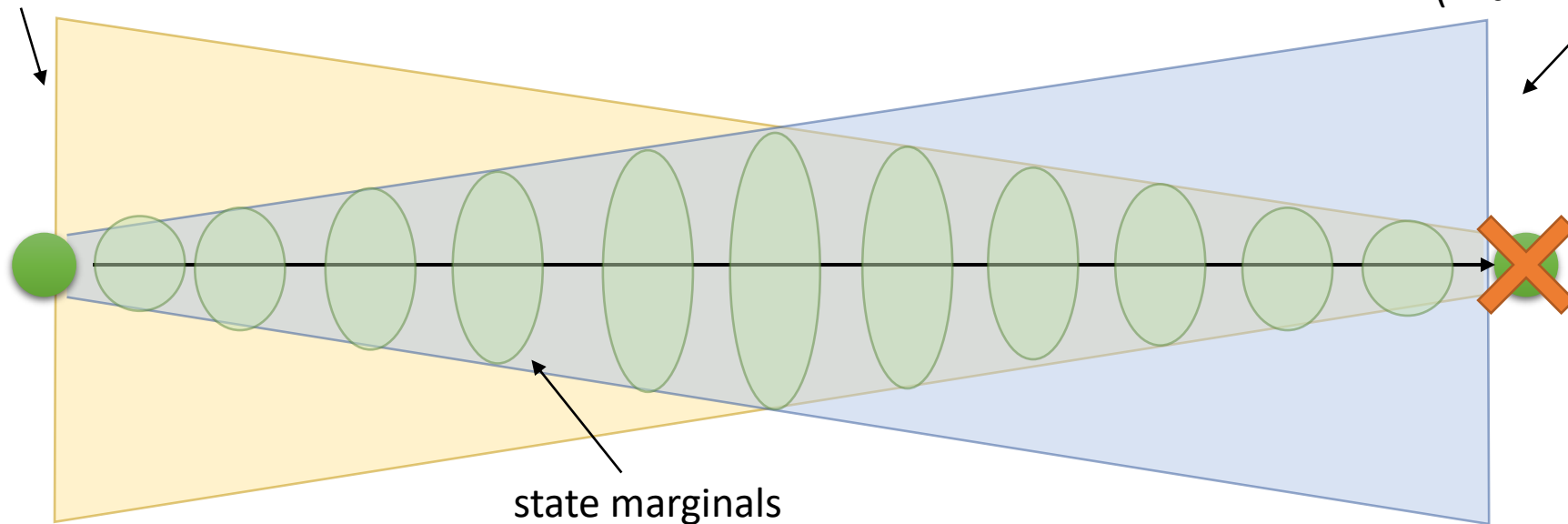
Li & Todorov, 2006



states with high probability of reaching goal

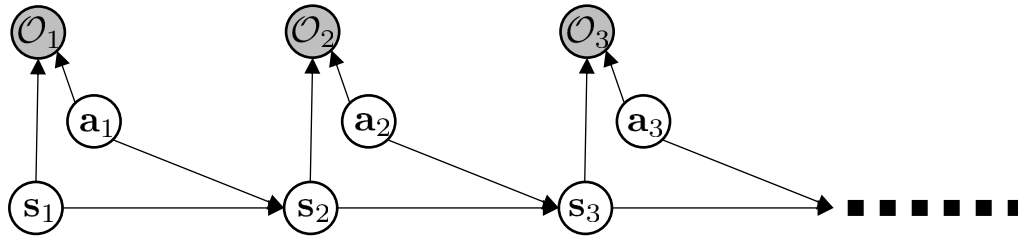
$$p(s_t) \propto \beta_t(s_t) \alpha_t(s_t)$$

states with high probability of being reached from initial state (with high reward)



Summary

1. Probabilistic graphical model for optimal control



2. Control = inference (similar to HMM, EKF, etc.)

3. Very similar to dynamic programming, value iteration, etc. (but “soft”)